Chiron

Workflow Reliability Consolidation and Emerging Architecture Pressure

Progress Report 4 • 10 Nov 2025 — 23 Nov 2025

Master’s Research Progress Update

Fahad Alothman

Progress Report 4

Title: Workflow Reliability Consolidation and Emerging Architecture Pressure
Reporting Window: 10 Nov 2025 — 23 Nov 2025
Submission Context: Master’s Research Progress Update

Abstract

Progress Report 4 documents a consolidation period focused on making the system reliably executable under real workflow conditions. The dominant work in this window was not a clean architecture reset, but practical hardening: workflow initialization stability, approval/rejection loop correctness, completion visibility, and more dependable UI/runtime behavior for methodology-driven execution.

At the same time, this period surfaced a strategic pressure that would shape later reports: mixed-language coordination repeatedly introduced implementation friction, making single-language project paths increasingly attractive for agent-driven development. This did not immediately conclude a full architecture transition, but it made the trade-off explicit and actionable for subsequent phases.

---

1. What Was Being Stabilized

During this window, implementation effort concentrated on making workflow execution trustworthy in day-to-day use.

The core stabilization targets were:

- workflow-init reliability,
- approval/rejection flow continuity,
- chronological coherence in execution/approval surfaces,
- and cleaner completion-state behavior.

This matters because the thesis value of guidance depends on execution fidelity. If workflows cannot be initialized, paused, approved, rejected, and resumed consistently, then methodological quality cannot be validated in practice.

---

2. Methodology Becoming Operational (Data-Driven in Runtime/UI)

A major shift in this period was moving methodology from static process definition toward operational runtime inputs.

In practical terms, the system increasingly treated options and guidance primitives as structured data that could be fetched, rendered, and acted on dynamically rather than hardcoded as one-off UI logic. This included improved handling of dynamic options, reusable option-card surfaces, and tighter linkage between workflow data and user-facing execution controls.

This marks a meaningful transition: parts of the methodology moved from static definition toward runtime-operational use inside the product.

---

3. Approval and Regeneration Loop Hardening

This period significantly improved approval-path resilience.

The execution model increasingly supported:

- clearer approval state progression,
- more stable rejection handling,
- better regeneration continuation behavior,
- and clearer temporal ordering of user-visible workflow events.

The result was a more inspectable workflow experience where users could follow decisions and outcomes with less ambiguity, reducing the need for constant manual reconstruction through chat.

---

4. Operational Model/Provider Stability Work

In parallel with workflow hardening, this window included practical model/provider stabilization work. The emphasis was not on introducing a final model strategy, but on reducing operational friction: provider/model selection consistency, safer fallback behavior, and fewer brittle integration points in runtime execution.

For the project narrative, this section should be read as reliability engineering rather than a separate architecture chapter. It supported the same core goal: make guided execution dependable enough for evaluation and continued use.

---

5. AX Capability Progression (Incremental, Not a Full Pivot)

AX-related capability (via the AX library/tooling path) increased in this period, especially around typed/structured usage paths and runtime integration behavior [2]. However, this should be framed as incremental capability growth inside the existing implementation stream, not as a standalone architecture pivot.

This distinction is important for historical accuracy: AX became more practically usable here, but the period's primary identity remained execution consolidation.

---

6. Process and Documentation Consolidation as Delivery Infrastructure

A substantial portion of the period's effort went into process and documentation consolidation: story handoffs, workflow context clarity, and artifact organization. Rather than treating this as overhead, this report treats it as delivery infrastructure.

For an agentic SDLC project, documentation quality directly affects implementation continuity. Better handoffs and clearer artifacts reduce context loss, improve repeatability, and increase the reliability of both agent-driven and human-supervised execution.

---

7. Derived Architecture Pressure at End of the Window

The most important strategic insight emerging from this period was not a finished architecture decision but a repeated implementation signal:

> repeated mixed-language coordination friction made single-language implementation paths increasingly attractive for agent-driven development.

As work intensified, maintaining cross-language coordination increased implementation friction and reduced execution smoothness. This created a clear pressure toward unifying more of the active implementation path in TypeScript.

This report therefore records the pressure and its basis, while leaving the formal architecture shift to subsequent progress reports where that shift becomes explicit and fully justified.

---

8. Open Questions Entering Progress Report 5

8.1 End-of-Period Target Stack Direction (Migration In Progress)

By the end of this reporting window, the project had not fully completed the transition, but it had begun moving toward a clear full-TypeScript target stack direction for the next phase.

The intended stack direction emerging at the end of Progress Report 4 was:

- **Desktop host:** Tauri [3]
- **Frontend:** React + TanStack Router + shadcn/ui
- **Backend/API:** TypeScript services with Hono + tRPC
- **AI orchestration/runtime path:** Mastra + AX [1], [2]
- **Data layer:** Drizzle ORM with PostgreSQL
- **Authentication:** Better Auth
- **Runtime/tooling:** Bun + Turborepo

At this stage, PostgreSQL was a deliberate choice because collaboration-oriented concerns were already active: shared state, multi-user continuity, and smoother team/project coordination were treated as important future requirements, not optional extras. This decision point is also relevant to later reports, where trade-offs in operational complexity, local-first ergonomics, and delivery constraints contributed to subsequent data-layer direction changes.

This stack definition is recorded here as a transition target derived from implementation pressure and execution experience in this period, not as a claim that all migration work was already complete inside this report window.

- How should the project formalize the transition from implementation pressure into an explicit stack decision?
- Which workflow reliability gains are stable enough to be treated as architectural assumptions?
- How can methodology-as-data continue to scale without overloading the UI/runtime complexity budget?
- What is the right sequencing for introducing larger platform changes while preserving delivery continuity?

These questions define the boundary between this consolidation report and the next phase.

Conclusion

Progress Report 4 should be understood as a reliability-consolidation window with strategic direction emerging from implementation reality. The dominant contribution was stabilizing

workflow execution under guided methodology. From that work, a clear architecture pressure became visible: recurring mixed-language coordination friction made single-language implementation paths increasingly attractive for agent-driven development.

This framing preserves historical continuity: the report captures what was hardened, what was learned, and why that learning mattered for the architecture decisions that followed.

References

[1] Mastra, “Mastra Documentation,” 2025. [Online]. Available: https://mastra.ai/docs
[2] axllm, “ax,” GitHub repository (`@ax-llm/ax`), 2025. [Online]. Available: https://github.com/axllm/ax
[3] Tauri Programme, “Tauri,” Documentation, 2025. [Online]. Available: https://tauri.app/